

**UNIVERSIDADE ESTADUAL DO SUDOESTE DA BAHIA  
CIÊNCIA DA COMPUTAÇÃO  
DESENVOLVIMENTO WEB 2026.1**

**Thiago Fernandes  
Vinícius de Oliveira**

**DOCUMENTO DE ARQUITETURA DO SISTEMA  
SISTEMA DE PRÉ-MATRÍCULA ACADÊMICA**

**Vitória da Conquista  
2026**

**THIAGO FERNANDES  
VINÍCIUS DE OLIVEIRA**

**DOCUMENTO DE ARQUITETURA DO SISTEMA  
SISTEMA DE PRÉ-MATRÍCULA ACADÊMICA**

Documento de especificação técnica e modelagem arquitetural contendo o detalhamento de camadas físicas e lógicas, fluxo transacional MVC, divisão por pacotes de domínio e rastreabilidade com os requisitos do Sistema de Pré-Matrícula Acadêmica. Projeto desenvolvido para a disciplina de Desenvolvimento Web do curso de Bacharelado em Ciência da Computação da Universidade Estadual do Sudoeste da Bahia (UESB).

**Vitória da Conquista  
2026**

## SUMÁRIO

|          |                                                                    |          |
|----------|--------------------------------------------------------------------|----------|
| <b>1</b> | <b>INTRODUÇÃO</b>                                                  | <b>2</b> |
| 1.1      | Objetivo . . . . .                                                 | 2        |
| 1.2      | Visão Geral da Solução . . . . .                                   | 2        |
| <b>2</b> | <b>DESCRIÇÃO DA ARQUITETURA UTILIZADA</b>                          | <b>2</b> |
| 2.1      | Padrão Model-View-Controller (MVC) Distribuído . . . . .           | 2        |
| 2.2      | Arquitetura Orientada a Domínios (Domain Package Design) . . . . . | 3        |
| <b>3</b> | <b>DIAGRAMA DE CAMADAS</b>                                         | <b>4</b> |
| 3.1      | Detalhamento Funcional das Camadas . . . . .                       | 4        |
| 3.1.1    | Camada de Apresentação (View/Frontend) . . . . .                   | 4        |
| 3.1.2    | Camada de Controle (Controller) . . . . .                          | 4        |
| 3.1.3    | Camada de Serviços (Service) . . . . .                             | 5        |
| 3.1.4    | Camada de Repositório (Repository) . . . . .                       | 5        |
| 3.1.5    | Camada de Domínio e Entidades (Entities) . . . . .                 | 5        |
| <b>4</b> | <b>ORGANIZAÇÃO E EMPILHAMENTO POR DOMÍNIO</b>                      | <b>6</b> |
| 4.1      | Benefícios da Modularização por Domínio . . . . .                  | 6        |
| <b>5</b> | <b>CONCLUSÃO</b>                                                   | <b>7</b> |

# 1 INTRODUÇÃO

## 1.1 Objetivo

Este documento descreve formalmente as decisões arquiteturais, o modelo de camadas lógicas e a organização estrutural adotada no Sistema de Pré-Matrícula Acadêmica da Universidade Estadual do Sudoeste da Bahia (UESB). O objetivo deste artefato é fornecer à equipe técnica, desenvolvedores e avaliadores acadêmicos uma compreensão profunda das fronteiras de responsabilidade do software, fluxo de dados e integrações tecnológicas adotadas.

## 1.2 Visão Geral da Solução

Para atender aos requisitos de alta manutenibilidade, segurança e desacoplamento operacional, o sistema foi projetado sobre uma **Arquitetura Cliente-Servidor Desacoplada**.

A camada de apresentação é implementada como uma *Single Page Application* (SPA) Angular independente, que se comunica de forma assíncrona por protocolo HTTP/JSON com uma API REST robusta construída sob o ecossistema Spring Boot. A persistência durável de dados é delegada ao SGBD relacional PostgreSQL, gerenciado em container Docker.

# 2 DESCRIÇÃO DA ARQUITETURA UTILIZADA

A arquitetura do sistema foi desenhada sob os pilares da **Clean Architecture** e da separação de responsabilidades (SOLID). O sistema se divide de maneira limpa entre o lado do cliente (Frontend SPA) e o lado do servidor (Backend Stateless API).

## 2.1 Padrão Model-View-Controller (MVC) Distribuído

Diferente das arquiteturas MVC monolíticas tradicionais, onde o servidor renderiza e despacha as telas de forma síncrona, este sistema adota um padrão de **MVC Distribuído**:

- **View (Visão):** Localizada integralmente no cliente por meio do framework Angular. A interface reage dinamicamente a mudanças de estado locais utilizando TypeScript, RxJS e folhas de estilo utilitárias do Tailwind CSS.
- **Controller (Controle):** Implementado no backend por meio do Spring Web (@RestController). É a barreira de entrada das requisições HTTP, responsável por validar a integridade estrutural das entradas utilizando Bean Validation (`jakarta.validation`) e despachar dados estruturados.
- **Model (Modelo):** Composto pelas entidades persistidas do JPA/Hibernate (Model/Entity) que representam o esquema do PostgreSQL, bem como pelas interfaces de repositório (Repository) que interagem com o banco de dados.

## 2.2 Arquitetura Orientada a Domínios (Domain Package Design)

Para evitar o acúmulo de arquivos em superpastas organizadas estritamente por tipos técnicos (padrão que gera alto acoplamento sistêmico), o backend utiliza o **Domain Driven Package Structure** (conforme RNF-02).

A aplicação é fragmentada em submódulos de negócio independentes (Ex: `student`, `discipline`, `enrollment`). Cada diretório de domínio encapsula seu próprio ecossistema de funcionamento (Controller, Service, Repository, DTOs e Entities). Essa abordagem garante:

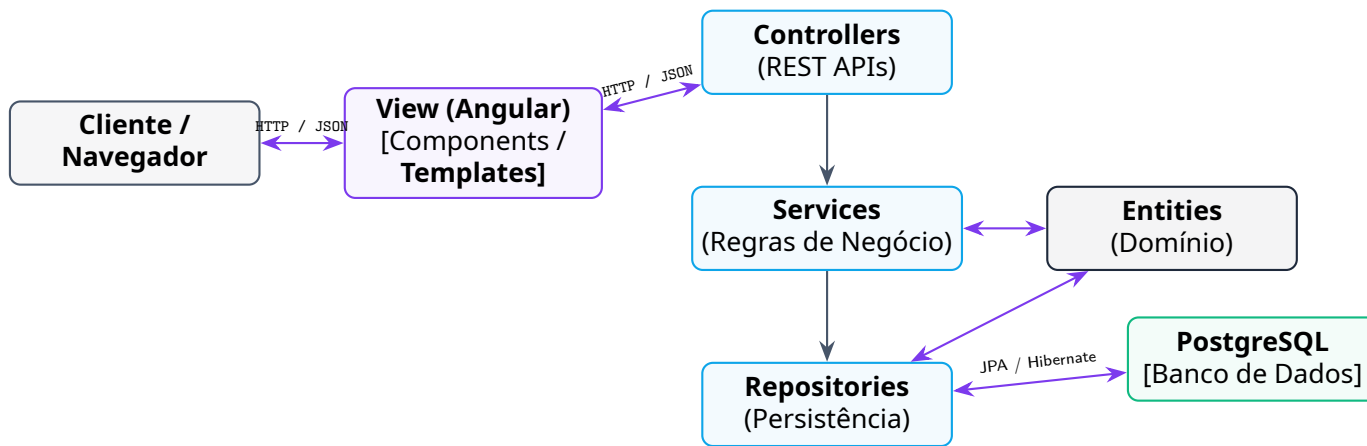
- **Alta Coesão:** Componentes que trabalham juntos para resolver um problema de negócio específico residem fisicamente no mesmo pacote.
- **Baixo Acoplamento:** Alterações na lógica interna de um domínio não propagam quebras de compilação em domínios paralelos. A comunicação entre domínios distintos ocorre por interfaces e contratos de serviços expostos (Services).

### 3 DIAGRAMA DE CAMADAS

O fluxo dinâmico de processamento de dados do sistema percorre camadas explícitas e isoladas por fronteiras lógicas.

A Figura 1 apresenta o diagrama de comunicação e fluxo transacional das camadas MVC distribuídas do sistema. O fluxo de informações segue um modelo de requisição e resposta sem estado (*stateless*), assegurado por assinatura criptográfica baseada em JSON Web Tokens (JWT).

Figura 1: Diagrama de comunicação física e lógica entre as camadas MVC



#### 3.1 Detalhamento Funcional das Camadas

##### 3.1.1 Camada de Apresentação (View/Frontend)

Representada pelo ecossistema Angular. Esta camada capta as intenções de cliques, digitação e interações físicas do estudante ou administrador. O frontend gerencia rotas preguiçosas (*lazy-loaded routes*) protegidas por guardas de segurança (*AuthGuard* e *RoleGuard*), impedindo o acesso visual a recursos administrativos por contas discentes. O *AuthInterceptor* anexa de forma transparente o token JWT armazenado em todas as chamadas AJAX direcionadas para o backend.

##### 3.1.2 Camada de Controle (Controller)

A camada de controle reside na API REST Spring Boot (*@RestController*). Ela intercepta requisições HTTP, decodifica payloads JSON e delega o processamento à camada de serviços correspondente.

Controllers são concebidos de forma enxuta (*thin controllers*), desprovidos de lógica de persistência ou decisões de negócio complexas. Suas responsabilidades limitam-se a:

- Mapeamento e resolução de rotas HTTP (GET, POST, PUT, DELETE);
- Validação sintática inicial dos payloads com anotações de validação de propriedade (*@NotNull*, *@Size*, etc.);

- Despacho de dados estruturados em Objetos de Transferência de Dados (DTOs).

### 3.1.3 *Camada de Serviços (Service)*

É o núcleo operacional do sistema, onde se centralizam as **\*\*Regras de Negócio (RN)\*\***. Os componentes anidados nesta camada de aplicação (@Service) coordenam a lógica de processamento de dados acadêmicos, regras de limites de matrícula (RN-041), checagem de vagas em tempo real (RN-047) e fechamento temporal de cronogramas.

A camada de serviços gerencia a demarcação transacional de banco de dados (@Transactional) garantindo atomicidade estrita nas escritas concorrentes e integridade dos dados e logs em caso de revertimentos em tempo de execução (*rollbacks*).

### 3.1.4 *Camada de Repositório (Repository)*

Atua como isolamento de persistência de dados. Baseia-se no framework Spring Data JPA, que estende interfaces genéricas de transação e geração automática de consultas do Hibernate. Os serviços acessam o banco de dados única e exclusivamente através desta camada de repositório, garantindo que mudanças no SGBD PostgreSQL não impactem de forma direta o código de negócio.

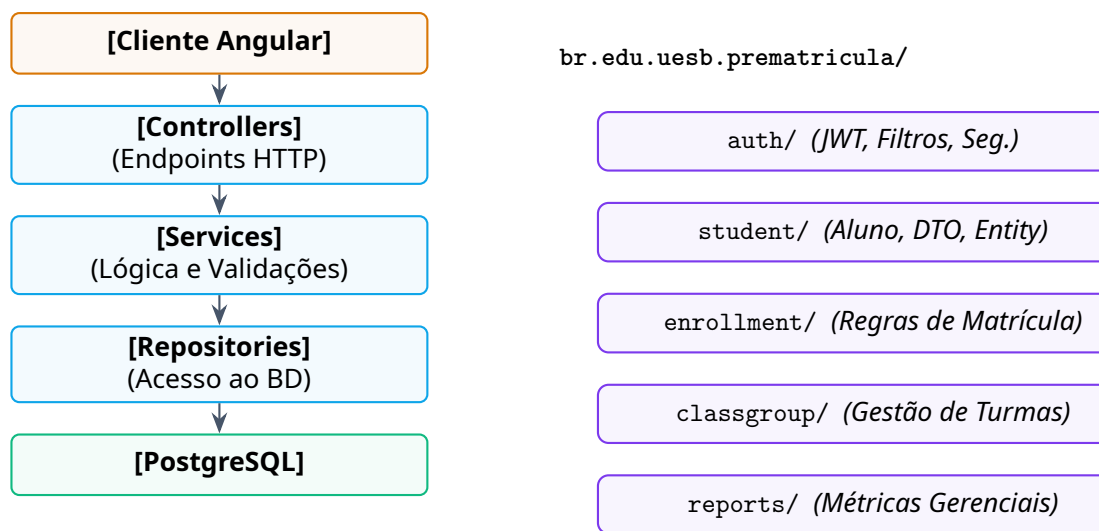
### 3.1.5 *Camada de Domínio e Entidades (Entities)*

Representada pelas classes Java mapeadas como entidades do banco pelo Hibernate. Estas classes refletem de forma fiel a modelagem lógica relacional do sistema de banco de dados (DER), definindo relacionamentos de integridade como um-para-um (@OneToOne) e um-para-muitos (@OneToMany), unidas aos metadados de chaves estrangeiras.

## 4 ORGANIZAÇÃO E EMPILHAMENTO POR DOMÍNIO

A Figura 2 descreve de maneira paralela o empilhamento vertical clássico de segurança e dados (à esquerda) em concomitância com o desacoplamento físico por domínios de negócio implementado no core do projeto (à direita). Essa estruturação garante que cada módulo seja autocontido.

Figura 2: Empilhamento lógico de tecnologias e divisão de pacotes por domínio



### 4.1 Benefícios da Modularização por Domínio

Ao alinhar a infraestrutura técnica com a modularização lógica por domínios de negócio, o sistema obtém vantagens técnicas essenciais no ciclo de vida de engenharia de software:

- **Desacoplamento Seguro:** O domínio de matrículas (`enrollment`) comunica-se com o domínio de alunos (`student`) por meio de interfaces seguras e de uso de DTOs, blindando e isolando as entidades internas do banco de dados (conforme RNF-21).
- **Evolução Simples de Regras de Negócio:** Se uma regra de processamento de turmas for alterada na instituição, apenas as classes internas do domínio `classgroup` necessitam passar por manutenção e validação de testes.
- **Facilidade de Escrita por Equipes Distribuídas:** Desenvolvedores podem trabalhar de forma simultânea e independente em domínios paralelos no repositório, mitigando drasticamente problemas de conflito de mesclagem no Git (*Merge Conflicts*).



## 5 CONCLUSÃO

A arquitetura do Sistema de Pré-Matrícula Acadêmica da Universidade Estadual do Sudoeste da Bahia (UESB) equilibra simplicidade e robustez. Ao descentralizar o padrão MVC clássico em duas aplicações autônomas e desacopladas, o sistema garante responsividade ágil na interface discente e robustez operacional na retaguarda.

Os diagramas de blocos vetoriais gerados neste documento delineiam as fronteiras e as portas de comunicação de requisições do sistema. A adoção dos pacotes divididos por domínios de negócio e o isolamento de persistência asseguram que o software está pronto para sofrer manutenção, evoluir e se integrar perfeitamente com os sistemas acadêmicos corporativos já estabelecidos, garantindo total conformidade técnica para o semestre letivo \*\*2026.1\*\*.